

DEVOPS - 06-03-2026

Flannel (CNI Add-on)

Flannel is a network plugin used in Kubernetes.

It allows Pods to communicate with each other across different nodes.

It creates a simple flat network across the cluster.

Each node gets a subnet and Flannel manages routing between nodes.

Reference:

<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

CLI Kubernetes Commands

1) kubectl version

Shows Kubernetes versions such as client, server and kustomize.

2) kubectl get nodes

Displays nodes available in the cluster and their status.

3) kubectl describe nodes kube-master

Shows detailed metadata of the node like CPU, memory, node IP and events.

Launching a Pod

4) kubectl run sietpod --image=httpd

Used to create or launch a Pod.

5) `kubectl get pods`

Used to view the Pods created.

6) `kubectl describe pod sietpod`

Shows metadata and detailed information about the Pod.

7) `kubectl exec -it sietpod -- /bin/bash`

Used to enter inside the container and execute commands.

8) `kubectl logs sietpod`

Used to check logs of the container.

9) `kubectl delete pod sietpod`

Deletes the Pod from the cluster.

Deployment

Deployment is used to manage Pods automatically.

It supports scaling, self healing and rolling updates.

1) `kubectl create deployment sietdeploy --image=httpd:latest`

Creates a deployment.

2) `kubectl get deployment`

Shows deployments available in the cluster.

3) `kubectl get pods`

Used to check Pods created by the deployment.

4) `kubectl delete pod <podname>`

If the Pod is deleted, Kubernetes automatically recreates it.

This feature is called Self Healing.

Scaling Deployment

6) `kubectl scale deployment sietdeploy --replicas=5`

Creates 5 Pods.

7) `kubectl create deployment test --image=nginx --replicas=3`

Creates a deployment with 3 Pods.

8) `kubectl scale deployment sietdeploy --replicas=2`

Reduces the number of Pods.

Delete Deployment

9) `kubectl delete deployment sietdeploy test`

Deletes the deployments.

10) `kubectl get pods`

Shows no Pods if deployment is deleted.

Rolling Update

11) `kubectl create deployment testdeploy --image=httpd --replicas=3`

12) `kubectl set image deployment testdeploy httpd=httpd:trixie`

Updates the container image.

13) `kubectl get pods`

Shows new Pods with updated image.

14) `kubectl rollout undo deployment testdeploy`

Reverts to previous version.

15) `kubectl rollout status deployment testdeploy`

Shows deployment update status.

Execute Pod

17) `kubectl exec -it <podname> /bin/bash`

Used to enter inside the running container.

Services

18) `kubectl get svc`

or

`kubectl get service`

Displays services in the cluster.

19) `kubectl expose deployment/testdeploy --type=NodePort --port=80`

Creates NodePort service.

Example output:

80:32145

80 = container port

32145 = NodePort used to access the application externally.

Testing the Service

21) `kubectl create deployment task --image=httpd --replicas=5`

22) `kubectl get deployment`

23) `kubectl get pods`

24) `kubectl expose deployment/task --type=NodePort --port=80`

25) Access using:

(worker-ip):(NodePort)

Extra Metadata

26) `kubectl get pods -o wide`

Shows extra information such as Pod IP and node name.

27) `kubectl get deployment -o wide`

Shows extra deployment details.

`-o wide` is used to display additional metadata.

Namespace

`kubectl get namespaces`

Displays all namespaces in the cluster.

Deployment YAML Example

`apiVersion: apps/v1`

`kind: Deployment`

`metadata:`

`name: nginx-deployment`

`spec:`

`selector:`

`matchLabels:`

`app: nginx`

```
replicas: 2
```

```
template:
```

```
  metadata:
```

```
    labels:
```

```
      app: nginx
```

```
  spec:
```

```
    containers:
```

```
      - name: nginx
```

```
        image: nginx:1.14.2
```

```
        ports:
```

```
          - containerPort: 80
```

Create YAML File

```
vi app-file.yaml
```

Apply YAML file

```
kubectl apply -f app-file.yaml
```